UNIT III – C++ DEBUGGING SOLUTIONS

Constructors, Destructors and Operator Overloading

Prepared from the uploaded Unit III question bank. Each problem includes the bug, root cause, corrected code, and expected behavior/output.

Question 1: Copy Constructor Pass-by-Value

Bug and root cause

The copy constructor uses DynamicString(DynamicString source). A copy constructor must take its own class type by reference. Passing by value requires the argument itself to be copied to initialize source, which calls the same copy constructor again. This recursive copy process cannot terminate. A typical compiler diagnostic is that the copy-constructor parameter must be a reference; exact wording varies by compiler.

Correct signature: DynamicString(const DynamicString& source). The reference avoids making another copy, and const allows copying from const objects and temporaries without modifying the source.

Corrected complete code

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char* buffer;
    int length;

public:
    DynamicString(const char* str) {
        length = static_cast<int>(std::strlen(str));
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    DynamicString(const DynamicString& source) {
        length = source.length;
        buffer = new char[length + 1];
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;
    s2.print();
    return 0;
}
```

Output:

```
Antigravity C++
```

Question 2: Shallow Copy and Double Free

Bug and root cause

The compiler-generated copy constructor copies each data member directly. For samples, it copies the pointer address, not the array elements. Therefore track.samples and buf.samples point to the same heap array. When buf is destroyed at the end of processAudio, it deletes the array; track then holds a dangling pointer. Reading it is undefined behavior, and track destruction attempts a second delete, causing a double-free/heap-corruption failure.

Line classification: missing deep-copy constructor; unsafe ownership of a raw heap pointer (Rule of Three issue).

Corrected complete code

```
#include <iostream>

class AudioBuffer {
public:
    int* samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = i * 10;
        std::cout << "Allocated buffer" << std::endl;
    }

    AudioBuffer(const AudioBuffer& other) : capacity(other.capacity) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i)
            samples[i] = other.samples[i];
        std::cout << "Deep copy allocated" << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Deallocating buffer" << std::endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);
    std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
    return 0;
}
```

Expected output (pointer addresses intentionally omitted because they vary):

```
Allocated buffer
Deep copy allocated
```

```
Processing sample 0: 0
Deallocating buffer
Back in main, sample 0: 0
Deallocating buffer
```

The two allocations are distinct: the original object owns one array and the copied parameter owns another. Each is deleted once.

Question 3: Stream Insertion Operator as a Member

Bug and root cause

A binary member operator expression is translated as left.operator@(right). Thus std::cout << f1 would require std::cout.operator<<(f1), not Fraction::operator<<(std::cout). The member function is attached to Fraction, so the natural stream expression does not match. Conversely, f1 << std::cout translates to f1.operator<<(std::cout), which matches the incorrect member signature but reverses normal stream syntax.

The stream insertion operator should be a non-member (often friend) taking std::ostream& first and const Fraction& second.

Corrected complete code

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    friend std::ostream& operator<<(std::ostream& os,
                                    const Fraction& fraction) {
        os << fraction.numerator << "/" << fraction.denominator;
        return os;
    }
};

int main() {
    Fraction f1(3, 4);
    std::cout << f1 << std::endl;
    return 0;
}
```

Output:

```
3/4
```

Question 4: Prefix vs. Postfix Increment

Correct signatures and meaning

Prefix: StepCounter& operator++() — increments the current object and returns it by reference, allowing chaining. Postfix: StepCounter operator++(int) — the dummy int distinguishes postfix; it returns a copy of the old value after incrementing the current object.

Corrected complete code

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    StepCounter& operator++() {       // Prefix ++obj
        ++steps;
        return *this;
    }

    StepCounter operator++(int) {     // Postfix obj++
        StepCounter old(*this);
        ++steps;
        return old;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);
    StepCounter c2 = ++c1;
    c2.display();
    c1.display();

    StepCounter c3 = c1++;
    c3.display();
    c1.display();
    return 0;
}
```

Output:

```
Steps: 11
Steps: 11
Steps: 11
Steps: 12
```

For prefix, c1 becomes 11 and c2 receives the updated value 11. For postfix, c3 receives the old value 11 while c1 becomes 12.

Question 5: Fundamental Operator Overloading Rules

Bug and root cause

The dot operator (.) cannot be overloaded. An operator overload must involve at least one user-defined type, so int operator+(int, int) is forbidden because both operands are built-in int values. C++ does not allow inventing a new token such as **; only existing operator tokens may be overloaded, and their precedence, associativity, and arity remain unchanged.

The commonly non-overloadable operators are . , .* , :: , ?: , and sizeof. (The question bank lists these.)

Corrected legal implementation

```
#include <iostream>

class MathVector {
public:
    double x, y;

    MathVector(double a = 0.0, double b = 0.0) : x(a), y(b) {}

    bool operator==(const MathVector& other) const {
        return x == other.x && y == other.y;
    }

    MathVector operator+(const MathVector& other) const {
        return MathVector(x + other.x, y + other.y);
    }

    MathVector power(int exponent) const {
        // Component-wise exponentiation
        double px = 1.0, py = 1.0;
        for (int i = 0; i < exponent; ++i) {
            px *= x;
            py *= y;
        }
        return MathVector(px, py);
    }
};

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);
    MathVector sum = v1 + v2;
    MathVector squared = v1.power(2);

    std::cout << (v1 == v2) << std::endl;
    std::cout << sum.x << ", " << sum.y << std::endl;
    std::cout << squared.x << ", " << squared.y << std::endl;
    return 0;
}
```

Output:

```
1
4, 6
4, 9
```

Note: power() is a normal member method, not an invented operator. This example defines component-wise exponentiation.

Quick Revision: Common Correct Forms

```
// Deep-copy constructor
ClassName(const ClassName& other);

// Prefix increment
ClassName& operator++();

// Postfix increment
ClassName operator++(int);

// Stream insertion
friend std::ostream& operator<<(std::ostream& os, const ClassName& obj);

// Stream extraction
friend std::istream& operator>>(std::istream& is, ClassName& obj);
```

Memory reminder: if a class owns memory allocated with new/new[], implement the appropriate copy behavior and destructor so objects do not share ownership accidentally. The uploaded question bank emphasizes the deep-copy constructor and destructor for these cases.